

CS 452/652 Winter 2025 - Lecture 4

ARM Processor, Context Switch

Jan 16, 2025 [prev](#)

LSA - [Learn the Architecture - A-profile](#)

- [AArch64 Exception model](#)
- [AArch64 Instruction Set Architecture \(ISA\)](#)

Context Switch

- context = registers + stack
 - registers: fast operands for processor operations
 - stack: dynamic storage for automatic variables
- stack switch
 - save registers (to somewhere) *without changing them*
 - stack switch: save/restore SP register
 - restore registers
- mode switch (privilege level) - processor exception
 - some state automatically saved
 - usually implies stack switch for safety & security
- system call & interrupt: stack switch & mode switch
- task vs. coroutine: scheduling

ARMv8 Basics

- RISC - 64-bit memory, but 32-bit instructions: opcode + operands => lots of addressing modes
 - immediate constants with shifting; pc-relative, etc., restricted
 - move, arithmetic, branch, load/store (index and/or increment)

ARMv8 Processor State

- execution state: 32 vs 64 bit mode (AArch32 vs. AArch64)
- exception level: 0...3 (cf. Intel/AMD "protection ring")
 - 0: *user program*
 - 1: *kernel*
 - 2: hypervisor (boot)
 - 3: secure monitor → example: trust/DRM
- security state: *normal* (non-secure) vs. secure world
- register file: X0 ... X30=LR, X31=XZR, PC, SP (EL 0-3)
 - program counter, banked stack pointer SP_ELx
- pstate: current processor status (implicit)
 - condition codes (N,Z,C,V), interrupt flags, execution state, exception level
 - Negative, Zero, Carry and oVerflow -> conditional branching
- named system registers; access with MRS/read, MSR/save
 - such as: ELR, ESR, SPSR, VBAR
- SP selector (SPSel): use current (banked) SP_ELx or SP_EL0
 - using SP_EL0 is unsafe

System Call

- synchronous exception - this is how a task asks the kernel for something
- dedicated instruction: `svc N`
- `ESR_EL1` holds exception code and `N`
- `ELR_EL1` holds return address (next PC after `svc`)
- `SPSR_EL1` hold pstate before exception
- processor in EL1 using `SP_EL1` (assuming `SPSel=1`)
- execution continues (PC) at hard-coded handler (exception vector)
- what needs to happen next?
 - save general-purpose registers
 - save `ELR_EL1`, `SPSR_EL1`, `SP_EL0`
 - we might not return to same task
 - restore kernel state
 - access system call arguments?
- resume user task:
 - save kernel state
 - restore `ELR_EL1`, `SPSR_EL1`, `SP_EL0`
 - return from exception: `eret`
 - restores PC from ELR, pstate from SPSR
 - returns to EL0 (then using `SP_EL0`)
- initialize user task: set up stack and "fake" context and resume

Exception Vector

- see LSA "Exception model", Table 4
- vectors: `VBAR_ELn` exceptions **to** EL `n`
 - 4 groups (**from** where): Current EL `SP0/x`, Lower EL 64/32
 - 4 vectors (32 instructions): Synchronous, IRQ, FIQ, Error
 - total of 2KB = $4 * 4 * 128$ bytes
- recommendation: set up dummy handlers for everything!